Celery & Redis – Complete Notes

Python | Django | Django REST Framework | Backend Development

Level: Beginner to Advanced

Purpose: Backend Development + Practical Projects + Interview Preparation

---

Table of Contents

1. Introduction to Celery
2. What is Redis?
3. Why Use Celery and Redis?
4. Celery Architecture
5. Important Celery Components
6. Redis Fundamentals
7. Installation and Setup
8. Django Project Configuration
9. Creating First Celery Task
10. Running Celery Worker
11. delay() vs apply_async()
12. Task States and Results
13. Celery with Django REST Framework
14. Sending Emails in Background
15. Task Retry Mechanism
16. Task Error Handling
17. Celery Beat and Scheduled Tasks
18. Redis Caching with Django
19. Celery Task Routing
20. Task Chain, Group, and Chord
21. Task Monitoring with Flower
22. Celery Production Configuration
23. Docker Integration
24. Redis Commands
25. Celery vs Redis vs RabbitMQ
26. Best Practices
27. Common Problems and Solutions
28. Practical Projects
29. Interview Questions and Answers
30. Learning Roadmap

---

1. Introduction to Celery

What is Celery?

Celery is an open-source distributed task queue system written in Python.

It allows developers to execute background tasks asynchronously or periodically.

In simple words:

Celery helps us execute time-consuming tasks in the background without blocking the main application.

Example

Suppose a user registers on a Django application.

After registration, we need to:

1. Create a user account.
2. Send a welcome email.
3. Send an SMS notification.
4. Generate a welcome PDF.
5. Create an activity log.

Without Celery, all operations may execute inside the same HTTP request.

This can make the API response slow.

With Celery, we can execute time-consuming operations in the background.

Without Celery

```
def register_user(request):
    user = User.objects.create(
        username="faruk",
        email="faruk@example.com"
    )

    send_email(user.email)

    return JsonResponse({
        "message": "Registration successful"
    })
```

Problem:

The HTTP request may wait until the email operation finishes.

With Celery

```
def register_user(request):
    user = User.objects.create(
        username="faruk",
        email="faruk@example.com"
    )

    send_email_task.delay(user.email)

    return JsonResponse({
        "message": "Registration successful"
    })
```

Now the email task can execute in the background.

---

2. What is Redis?

Definition

Redis stands for Remote Dictionary Server.

Redis is an in-memory data store that supports key-value data structures.

It is commonly used for:

- Caching
- Message brokering
- Session storage
- Real-time applications
- Rate limiting
- Temporary data storage
- Pub/Sub messaging
- Leaderboards

Redis Example

```
SET username "Faruk"

GET username
```

Output:

```
"Faruk"
```

Another example:

```
SET age 25

GET age
```

Output:

```
"25"
```

Redis stores data in memory, making data access very fast in many workloads.
Redis also supports persistence, depending on its configuration.

---

3. Why Use Celery and Redis?

Celery and Redis solve different problems.
Celery handles task execution.
Redis can act as the message broker that stores task messages until workers consume them.

Common Use Cases

1. Background email sending.
2. SMS notifications.
3. PDF generation.
4. Image processing.
5. Video processing.
6. Bulk data processing.
7. Scheduled database cleanup.
8. External API requests.
9. Report generation.
10. Data synchronization.
11. Order processing.
12. Payment-related background workflows.

Why not execute everything in Django views?

Because long-running operations can:

- Increase response time.
- Cause request timeouts.
- Consume web server resources.
- Reduce application throughput.
- Make the user experience slower.

---

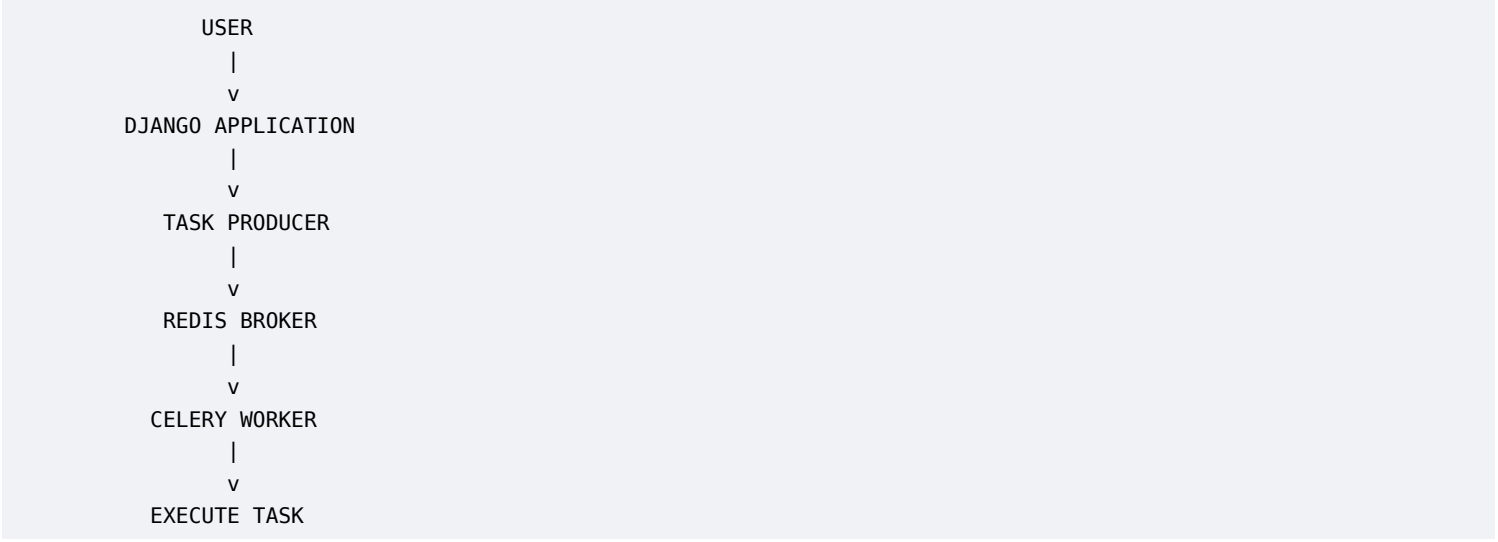
4. Celery Architecture

How Celery Works

The basic architecture contains:

1. Django Application
2. Producer
3. Message Broker
4. Celery Worker
5. Result Backend
6. Celery Beat

Architecture Diagram



```
                |
                v
         RESULT BACKEND
                |
                v
           TASK RESULT
```

Component Responsibilities

Component	Responsibility
Django	Creates background tasks
Producer	Sends task messages
Redis Broker	Stores queued messages
Celery Worker	Executes tasks
Result Backend	Stores task results
Celery Beat	Schedules periodic tasks

Important

Redis can work as both:

- Message Broker
- Result Backend

These are separate logical responsibilities.

---

5. Important Celery Components

5.1 Task

A task is a Python function that Celery can execute.

Example:

```
from celery import shared_task

@shared_task
def add_numbers(a, b):
    return a + b
```

5.2 Producer

A producer sends tasks to the broker.

Example:

```
add_numbers.delay(10, 20)
```

A Django application commonly acts as the producer.

5.3 Broker

A broker transports messages between producers and workers.

Examples:

- Redis
- RabbitMQ

5.4 Worker

A worker receives and executes background tasks.

Command:

```
celery -A config worker --loglevel=info
```

5.5 Result Backend

Stores task execution results and states when configured.

Example:

```
CELERY_RESULT_BACKEND = "redis://localhost:6379/1"
```

5.6 Celery Beat

Celery Beat schedules periodic tasks.

Example:

- Every 10 minutes.
- Every hour.
- Every day at 9 AM.

---

6. Redis Fundamentals

6.1 Redis Data Structures

Redis supports different data structures.

Data Structure	Description
String	Key-value data
List	Ordered collection
Set	Unique values
Sorted Set	Ordered values with scores
Hash	Field-value collection
Stream	Append-only event data

6.2 Redis String

```
SET name "Faruk"

GET name

DEL name
```

6.3 Redis List

```
LPUSH users "Faruk"

RPUSH users "Omar"

LRANGE users 0 -1
```

6.4 Redis Set

```
SADD skills "Python"

SADD skills "Django"

SMEMBERS skills
```

6.5 Redis Hash

```
HSET user:1 name "Faruk"

HSET user:1 age 25

HGET user:1 name

HGETALL user:1
```

6.6 Redis Expiration

```
SET otp 123456

EXPIRE otp 60

TTL otp
```

The key expires after 60 seconds.

---

7. Installation and Setup

7.1 Install Python Packages

```
pip install celery redis
```

For Django Redis caching:

```
pip install django-redis
```

For monitoring:

```
pip install flower
```

7.2 Install Redis

Using Docker:

```
docker run -d \
    --name redis \
    -p 6379:6379 \
    redis:7
```

Check Redis:

```
docker exec -it redis redis-cli ping
```

Expected output:

```
PONG
```

7.3 Check Redis Connection with Python

```
import redis

client = redis.Redis(
    host="localhost",
    port=6379,
    db=0,
    decode_responses=True
)

print(client.ping())
```

Output:

```
True
```

---

8. Django Project Configuration

Example project structure:

```
myproject/
│
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   ├── wsgi.py
```

```
│   └── celery.py
│
├── users/
│   ├── migrations/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   └── tasks.py
│
└── manage.py
```

Step 1: Create celery.py

File:
`config/celery.py`

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)

app = Celery("config")

app.config_from_object(
    "django.conf:settings",
    namespace="CELERY"
)

app.autodiscover_tasks()
```

Explanation

`os.environ.setdefault()`
Sets the Django settings module.
`Celery("config")`
Creates the Celery application.
`config_from_object()`
Loads configuration from Django settings.
`namespace="CELERY"`
Loads settings that begin with CELERY_.
`autodiscover_tasks()`
Automatically discovers tasks.py files in installed Django applications.

Step 2: Update init.py

File:
`config/__init__.py`

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

This ensures that the Celery application is loaded with the Django project.

Step 3: Configure settings.py

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"

CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/1"

CELERY_ACCEPT_CONTENT = ["json"]

CELERY_TASK_SERIALIZER = "json"
```

```
CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Dhaka"

CELERY_ENABLE_UTC = True
```

Configuration Explanation

Setting	Purpose
CELERY_BROKER_URL	Broker connection
CELERY_RESULT_BACKEND	Result storage
CELERY_ACCEPT_CONTENT	Accepted formats
CELERY_TASK_SERIALIZER	Task serialization
CELERY_RESULT_SERIALIZER	Result serialization
CELERY_TIMEZONE	Timezone
CELERY_ENABLE_UTC	UTC support

Redis database:

- Database 0: Broker
- Database 1: Result backend

These are logical Redis databases, not separate Redis servers.

---

9. Creating Your First Celery Task

Create:
`users/tasks.py`

```
from celery import shared_task

@shared_task
def add_numbers(a, b):
    result = a + b
    return result
```

Open Django shell:

```
python manage.py shell
```

Execute:

```
from users.tasks import add_numbers

result = add_numbers.delay(10, 20)

print(result.id)
```

The task ID identifies the submitted task.
Check the result:

```
print(result.get(timeout=10))
```

Output:

```
30
```

Important

`.delay()` sends the task to the broker.
`.get()` waits for the result.
Avoid calling `.get()` inside normal Django API requests because it can block the request.

---

10. Running Celery Worker

Open another terminal.

```
celery -A config worker --loglevel=info
```

For Windows development:

```
celery -A config worker --loglevel=info --pool=solo
```

Worker Responsibilities

1. Connects to the broker.
2. Receives task messages.
3. Executes registered tasks.
4. Reports task status.
5. Stores results if a result backend is configured.

Useful Commands

Start worker:

```
celery -A config worker -l info
```

Start worker with concurrency:

```
celery -A config worker -l info --concurrency=4
```

Inspect active tasks:

```
celery -A config inspect active
```

Inspect registered tasks:

```
celery -A config inspect registered
```

---

11. delay() vs apply_async()

Both methods send tasks to the broker.

delay()

Simple task execution:

```
send_email.delay("user@example.com")
```

apply_async()

Supports additional execution options.

```
send_email.apply_async(
    args=["user@example.com"],
    countdown=10
)
```

The task is scheduled for execution after approximately 10 seconds.

Custom Queue

```
send_email.apply_async(
    args=["user@example.com"],
    queue="emails"
)
```

ETA

```
from datetime import datetime, timedelta, timezone

send_email.apply_async(
    args=["user@example.com"],
    eta=datetime.now(timezone.utc) + timedelta(minutes=5)
)
```

Comparison

Feature	delay()	apply_async()
Basic execution	Yes	Yes
Countdown	No	Yes
ETA	No	Yes
Custom queue	No	Yes
Custom execution options	Limited	Yes

---

12. Task States and Results

Celery provides task states.

State	Meaning
PENDING	Waiting or state unknown
STARTED	Execution started
SUCCESS	Execution completed
FAILURE	Execution failed

RETRY	Waiting for retry
REVOKED	Task revoked

Example:

```
result = add_numbers.delay(10, 20)

print(result.id)

print(result.status)

print(result.ready())

print(result.successful())
```

Task Result

```
print(result.get(timeout=10))
```

Check Task Failure

```
if result.failed():
    print("Task failed")
```

Note: Task status visibility depends on result backend configuration and task tracking settings.

---

13. Celery with Django REST Framework

Example: User registration with background email.

tasks.py

```
from celery import shared_task

from django.core.mail import send_mail


@shared_task
def send_welcome_email(email):

    send_mail(
        subject="Welcome to Our Platform",
        message="Thank you for registering.",
        from_email="admin@example.com",
        recipient_list=[email],
        fail_silently=False
    )

    return "Email sent successfully"
```

views.py

```
from django.db import transaction

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from .models import User
from .tasks import send_welcome_email


class RegisterAPIView(APIView):
```

```
    def post(self, request):

        user = User.objects.create(
            username=request.data["username"],
            email=request.data["email"]
        )

        transaction.on_commit(
            lambda: send_welcome_email.delay(user.email)
        )

        return Response(
            {
                "message": "Registration successful"
            },
            status=status.HTTP_201_CREATED
        )
```

Why transaction.on_commit()?

It ensures that the task is dispatched only after the database transaction commits successfully.
Otherwise, a worker might try to access data before the database transaction is committed.
For production applications, use serializers and proper validation rather than directly trusting request data.

---

14. Sending Emails in Background

Configure Email Settings

```
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"

EMAIL_HOST = "smtp.gmail.com"

EMAIL_PORT = 587

EMAIL_USE_TLS = True

EMAIL_HOST_USER = "your-email@gmail.com"

EMAIL_HOST_PASSWORD = "your-app-password"

DEFAULT_FROM_EMAIL = EMAIL_HOST_USER
```

Never commit real credentials to GitHub.
Use environment variables or a secret manager.

Email Task

```
from celery import shared_task

from django.core.mail import send_mail


@shared_task
def send_email_task(email, subject, message):

    send_mail(
        subject,
        message,
        None,
        [email],
        fail_silently=False
    )

    return "Email sent"
```

Call Task

```
send_email_task.delay(
    "user@example.com",
    "Welcome",
    "Welcome to our application"
)
```

---

15. Task Retry Mechanism

Retry allows a failed task to execute again.
Example:

```
from celery import shared_task


@shared_task(
    bind=True,
    max_retries=3,
    default_retry_delay=10
)
def send_email_task(self, email):

    try:

        # Email sending logic
        print(f"Sending email to {email}")

        return "Success"

    except Exception as exc:

        raise self.retry(exc=exc)
```

Explanation

Parameter	Description
bind=True	Provides task instance
max_retries=3	Maximum retry count
default_retry_delay=10	Retry delay in seconds
self.retry()	Requests task retry

Automatic Retry

```
@shared_task(
    autoretry_for=(ConnectionError,),
    retry_kwargs={"max_retries": 3},
    retry_backoff=True
)
def external_api_task():
```

```
    # External API request
    pass
```

Exponential Backoff

Exponential backoff increases the delay between retries.

Example:

```
Retry 1: 2 seconds
Retry 2: 4 seconds
Retry 3: 8 seconds
Retry 4: 16 seconds
```

The actual delay depends on configuration and limits.

---

16. Task Error Handling

Example:

```
from celery import shared_task


@shared_task(bind=True)
def process_payment(self, order_id):

    try:

        # Payment processing logic
        return "Payment processed"

    except ConnectionError as exc:

        raise self.retry(
            exc=exc,
            countdown=10,
            max_retries=3
        )

    except Exception:

        raise
```

Important Concepts

- Exception handling
- Retry policies
- Logging
- Task time limits
- Idempotency
- Dead-letter handling where supported by the broker architecture

What is Idempotency?

An idempotent operation produces the same intended final state even if executed multiple times.

Example:

A payment task should not charge the customer twice when retried.

Use:

- Unique transaction IDs.
- Database constraints.
- Idempotency keys.
- Transaction-safe updates.

---

17. Celery Beat and Scheduled Tasks

Celery Beat is a scheduler for periodic tasks.

Examples:

- Daily report generation.

- Hourly database cleanup.
- Periodic notifications.
- Expired session cleanup.

Create Task

`users/tasks.py`

```
from celery import shared_task


@shared_task
def clean_old_data():

    print("Cleaning old data")

    return "Cleanup completed"
```

Configure Beat

`settings.py`

```
from celery.schedules import crontab


CELERY_BEAT_SCHEDULE = {

    "clean-data-every-10-minutes": {

        "task": "users.tasks.clean_old_data",

        "schedule": 600.0,

    },

}
```

Run Beat

```
celery -A config beat --loglevel=info
```

Run Worker

```
celery -A config worker --loglevel=info
```

Beat and worker are normally separate processes.

Crontab Examples

Every minute:

```
crontab()
```

Every hour:

```
crontab(minute=0)
```

Every day at 9 AM:

```
crontab(hour=9, minute=0)
```

Every Monday:

```
crontab(day_of_week=1, hour=9, minute=0)
```

Every month on the first day:

```
crontab(day_of_month=1, hour=0, minute=0)
```

Important

Run only one scheduler instance for a given schedule unless you intentionally design for distributed scheduling.
Duplicate schedulers can enqueue duplicate tasks.

---

18. Redis Caching with Django

Redis is also useful for caching.

Install:

```
pip install django-redis
```

Configure `settings.py`:

```
CACHES = {

    "default": {

        "BACKEND": "django_redis.cache.RedisCache",

        "LOCATION": "redis://127.0.0.1:6379/2",

        "OPTIONS": {

            "CLIENT_CLASS": "django_redis.client.DefaultClient",

        }

    }

}
```

Set Cache

```
from django.core.cache import cache

cache.set(
    "username",
    "Faruk",
    timeout=300
)
```

Get Cache

```
username = cache.get("username")

print(username)
```

Delete Cache

```
cache.delete("username")
```

Check Cache

```
if cache.get("username"):

    print("Cache exists")
```

Cache Use Cases

- User profile caching.
- Product listing caching.
- API response caching.
- Frequently accessed database queries.
- Session management.

Cache Expiration

```
cache.set("otp", "123456", timeout=60)
```

The value expires after 60 seconds.

---

19. Celery Task Routing

Task routing allows tasks to be sent to different queues.
Example:

```
CELERY_TASK_ROUTES = {

    "users.tasks.send_email_task": {
        "queue": "emails"
    },

    "users.tasks.generate_report": {
        "queue": "reports"
    },

}
```

Start Email Worker

```
celery -A config worker -Q emails -l info
```

Start Report Worker

```
celery -A config worker -Q reports -l info
```

Why Use Multiple Queues?

Different tasks have different resource requirements.
Example:

Queue	Task
emails	Email sending
reports	PDF/report generation
payments	Payment-related operations
default	General tasks

This helps isolate workloads.

---

20. Task Chain, Group, and Chord

Celery provides workflows for coordinating multiple tasks.

20.1 Chain

Chain executes tasks sequentially.

```
from celery import chain


workflow = chain(

    task_one.s(),

    task_two.s(),

    task_three.s()

)

workflow.apply_async()
```

Output of one task is passed to the next task by default.

20.2 Group

Group executes multiple tasks independently and concurrently.

```
from celery import group


workflow = group(

    add_numbers.s(10, 20),

    add_numbers.s(30, 40),

    add_numbers.s(50, 60)

)

result = workflow.apply_async()
```

20.3 Chord

Chord executes a callback after all group tasks finish successfully.

```
from celery import chord, group


workflow = chord(

    group(

        add_numbers.s(10, 20),

        add_numbers.s(30, 40)

    ),

    process_results.s()

)

workflow.apply_async()
```

Comparison

Feature	Chain	Group	Chord
Sequential	Yes	No	No
Parallel	No	Yes	Yes
Callback	Sequential next task	No built-in callback	Yes
Use case	Workflow	Independent jobs	Aggregate results

---

21. Task Monitoring with Flower

Flower is a web-based monitoring tool for Celery.

Install:

```
pip install flower
```

Run:

```
celery -A config flower
```

Open:

```
http://localhost:5555
```

Flower Features

- Worker monitoring.
- Active task monitoring.
- Completed task monitoring.
- Failed task monitoring.
- Task execution time.
- Worker status.
- Task history.

Flower is especially useful during development and production monitoring.
Secure its web interface before exposing it publicly.

---

22. Celery Production Configuration

Example:

```
CELERY_TASK_TRACK_STARTED = True

CELERY_TASK_TIME_LIMIT = 300

CELERY_TASK_SOFT_TIME_LIMIT = 240

CELERY_WORKER_PREFETCH_MULTIPLIER = 1

CELERY_TASK_ACKS_LATE = True

CELERY_TASK_REJECT_ON_WORKER_LOST = True

CELERY_WORKER_MAX_TASKS_PER_CHILD = 1000
```

Configuration Explanation

Setting	Purpose
TASK_TRACK_STARTED	Track task start
TASK_TIME_LIMIT	Hard execution time limit
TASK_SOFT_TIME_LIMIT	Soft execution time limit
WORKER_PREFETCH_MULTIPLIER	Controls task prefetch
TASK_ACKS_LATE	Acknowledge after execution
TASK_REJECT_ON_WORKER_LOST	Requeue on certain worker losses
WORKER_MAX_TASKS_PER_CHILD	Restart worker child after task count

These settings should be selected according to workload.
Late acknowledgement can cause tasks to execute more than once after failures. Tasks must be designed to tolerate duplicates.

---

23. Docker Integration

Example `docker-compose.yml`:

```
services:

  redis:
    image: redis:7
    container_name: redis
    ports:
      - "6379:6379"

  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    depends_on:
      - redis

  celery:
    build: .
    command: celery -A config worker -l info
    volumes:
      - .:/app
    depends_on:
      - redis

  celery-beat:
    build: .
    command: celery -A config beat -l info
    volumes:
      - .:/app
    depends_on:
      - redis
```

Inside Docker Compose, use the Redis service name:

```
CELERY_BROKER_URL = "redis://redis:6379/0"

CELERY_RESULT_BACKEND = "redis://redis:6379/1"
```

Start:

```
docker compose up --build
```

Note: `depends_on` controls startup order but does not guarantee Redis is ready to accept connections. Production deployments should use health checks and appropriate restart policies.

---

24. Important Redis Commands

Command	Description
PING	Check connection
SET	Store value
GET	Retrieve value
DEL	Delete key
EXISTS	Check key
EXPIRE	Set expiration
TTL	Get remaining expiration

INCR	Increment integer
DECR	Decrement integer
LPUSH	Add to list beginning
RPUSH	Add to list end
LRANGE	Get list elements
SADD	Add set element
SMEMBERS	Get set members
HSET	Set hash field
HGET	Get hash field
HGETALL	Get entire hash
FLUSHDB	Clear current database
FLUSHALL	Clear all databases

Important Warning

Avoid using `FLUSHDB` and `FLUSHALL` in production unless you fully understand their effects.

---

25. Celery vs Redis vs RabbitMQ

Feature	Celery	Redis	RabbitMQ
Type	Task queue framework	In-memory data store	Message broker
Background execution	Yes	No, by itself	No, by itself
Caching	No	Yes	No
Message brokering	Uses a broker	Yes	Yes
Python support	Excellent	Excellent	Via clients
Task scheduling	Beat	Not inherently	Not inherently

Important Difference

Celery is a task execution framework.
Redis and RabbitMQ are broker options.
Celery can use Redis or RabbitMQ as its message broker.

---

26. Best Practices

1. Keep tasks small and focused.
2. Avoid blocking Django API requests.
3. Use retries for temporary failures.
4. Make tasks idempotent.
5. Use appropriate task time limits.
6. Monitor worker health.

7. Use separate queues for different workloads.
8. Avoid unnecessary result storage.
9. Use `transaction.on_commit()` when dispatch depends on database commits.
10. Keep secrets in environment variables.
11. Protect Redis from unauthorized access.
12. Configure Redis persistence where needed.
13. Use proper logging.
14. Monitor task failures.
15. Avoid running duplicate Beat schedulers.
16. Use JSON serialization where practical.
17. Avoid passing Django model instances directly to tasks.
18. Pass model IDs and retrieve objects inside the task.
19. Handle duplicate task execution safely.
20. Configure worker concurrency based on CPU and workload.

---

27. Common Problems and Solutions

Problem 1: Connection Refused

Error:

```
redis.exceptions.ConnectionError
```

Possible solutions:

- Check whether Redis is running.
- Check host and port.
- Verify Docker port mapping.
- Verify the broker URL.

Problem 2: Task Not Registered

Error:

```
Received unregistered task
```

Possible solutions:

- Check `app.autodiscover_tasks()`.
- Check Django `INSTALLED_APPS`.
- Verify the task import path.
- Restart workers.

Problem 3: Task Remains PENDING

Possible causes:

- Worker is not running.
- Task was not consumed.
- Result backend is unavailable.
- Task state is not recorded.

Check worker logs and broker connectivity.

Remember: PENDING can also mean that the task state is unknown.

Problem 4: Task Executes Twice

Possible causes:

- Worker crash.
- Late acknowledgement.
- Retry.
- Duplicate task submission.

Solution:

Implement idempotency and database-level safeguards.

Problem 5: Redis Memory Usage Is High

Possible solutions:

- Set appropriate expiration.

- Review cache policies.
- Avoid unnecessary result storage.
- Monitor Redis memory.
- Configure suitable eviction and persistence policies.

---

28. Practical Projects

Project 1: Background Email System

Tech Stack:

- Python
- Django
- Django REST Framework
- Celery
- Redis

Features:

- User registration.
- Welcome email.
- Password reset email.
- Bulk email.
- Email retry.
- Task status.

Project 2: Background PDF Generator

Features:

- Generate PDF.
- Store generated files.
- Background processing.
- Task progress tracking.
- Download generated PDF.

Project 3: E-commerce Order Processing

Features:

- Order creation.
- Order confirmation email.
- Inventory update.
- Background notification.
- Order status tracking.
- Retry failed operations.

Project 4: Scheduled Data Cleanup

Features:

- Celery Beat.
- Expired data cleanup.
- Periodic database maintenance.
- Automated notifications.
- Logging.

Project 5: Image Processing System

Features:

- Image upload.
- Background image resizing.
- Thumbnail generation.
- Image compression.
- Task status.

---

29. Interview Questions and Answers

Q1. What is Celery?

Celery is a distributed task queue used to execute asynchronous and scheduled background tasks in Python applications.

Q2. What is Redis?

Redis is an in-memory data store commonly used for caching, message brokering, and temporary data storage.

Q3. Why do we use Celery with Django?

We use Celery to execute time-consuming tasks in the background without blocking HTTP requests.

Q4. What is a message broker?

A message broker transfers messages between producers and consumers.

Q5. What is a Celery worker?

A Celery worker is a process that receives and executes background tasks.

Q6. What is Celery Beat?

Celery Beat is a scheduler that periodically sends tasks to the broker.

Q7. What is a result backend?

A result backend stores task execution states and results.

Q8. What is the difference between delay() and apply_async()?

`delay()` is a shortcut for sending tasks. `apply_async()` provides additional execution options such as ETA, countdown, and queue routing.

Q9. Can Redis work as both broker and result backend?

Yes. Redis can serve both roles using separate logical databases or configurations.

Q10. What is task retry?

Task retry allows a failed task to execute again.

Q11. What is task idempotency?

Idempotency means repeated execution produces the same intended final state.

Q12. What is Flower?

Flower is a web-based monitoring tool for Celery workers and tasks.

Q13. What is the difference between synchronous and asynchronous execution?

Synchronous execution waits for an operation to complete. Asynchronous execution allows other work to proceed while an operation is running.

Q14. What is task routing?

Task routing sends specific tasks to designated queues.

Q15. What is the difference between Chain, Group, and Chord?

- Chain: Sequential execution.
- Group: Parallel execution.
- Chord: Parallel execution followed by a callback.

Q16. What happens if a Celery worker crashes?

Depending on acknowledgement settings and broker behavior, tasks may be redelivered, remain unacknowledged, or be lost in some configurations. Idempotency is essential.

Q17. Why use transaction.on_commit()?

It ensures task dispatch occurs after a successful database transaction commit.

Q18. Why should we avoid calling result.get() in a Django view?

Because it blocks the HTTP request while waiting for the background task result.

Q19. What is Redis caching?

Redis caching stores frequently accessed data in memory to reduce repeated database queries and improve response time.

Q20. Can Celery work without Redis?

Yes. Celery supports other brokers, including RabbitMQ.

---

30. Learning Roadmap

Phase 1: Fundamentals

- Understand background processing.
- Learn synchronous vs asynchronous execution.

- Understand task queues.
- Learn Redis basics.

Phase 2: Redis

- Redis installation.
- Key-value operations.
- Lists, Sets, Hashes.
- Expiration and TTL.
- Redis Python client.
- Django caching.

Phase 3: Celery

- Celery installation.
- Celery configuration.
- Creating tasks.
- Running workers.
- delay() and apply_async().
- Task results and states.

Phase 4: Django Integration

- Celery with Django.
- Celery with DRF.
- Background email.
- Background PDF generation.
- Database transaction handling.

Phase 5: Advanced Celery

- Retry.
- Error handling.
- Task routing.
- Chain.
- Group.
- Chord.
- Celery Beat.

Phase 6: Production

- Docker.
- Flower.
- Worker concurrency.
- Logging.
- Task idempotency.
- Redis security.
- Monitoring.

Phase 7: Portfolio Project

Build a complete Django REST API with:

- User authentication.
- Background email.
- Scheduled tasks.
- Redis caching.
- Task status tracking.
- Retry mechanism.
- Flower monitoring.
- Docker deployment.

---

31. Official Documentation

1. Celery Documentation: https://docs.celeryq.dev/en/stable/
2. Redis Documentation: https://redis.io/docs/latest/

3. Celery with Django: https://docs.celeryq.dev/en/stable/django/first-steps-with-django.html
4. Flower Documentation: https://flower.readthedocs.io/en/latest/
5. Django Documentation: https://docs.djangoproject.com/en/stable/
6. Celery GitHub: https://github.com/celery/celery
7. Redis GitHub: https://github.com/redis/redis

Final Advice

For a Python Django Backend Developer, Celery and Redis are valuable skills.

Do not focus only on theoretical concepts.

Build at least one real-world project using:

Python + Django REST Framework + PostgreSQL + Celery + Redis + Docker

Implement:

- Background email sending.
- Scheduled tasks.
- Task retries.
- Redis caching.
- Task status tracking.
- Error handling.
- Monitoring.

These practical skills will help you understand how background processing works in real-world backend applications.